

PLATAFORMA DE AUTOMAÇÃO COM IA · IRKO

Documento de *Arquitetura*

Plataforma low-code que transforma uma descrição em linguagem natural em automações executáveis e publicáveis, com assistente de IA, runtime isolado e aplicações internas para usuários não técnicos.

VERSÃO DA API

1.0.0

ESCOPO

BACKEND FASTAPI · FRONTEND
REACT · RUNTIME

DATA DE REFERÊNCIA

JUNHO DE 2026

ORIGEM

CÓDIGO-FONTE, BRANCH MAIN

Sumário

01	Visão geral e stack tecnológico	
02	Arquitetura de alto nível	
03	Frontend: os dois modos de uso	
04	Backend: API e routers	
05	Persistência e modelo de dados	
06	Runtime de execução de automações	
07	Camada de inteligência artificial	
08	Apps publicados e controle de acesso	
09	Fluxos principais (ponta a ponta)	
10	Decisões de arquitetura e trade-offs	
11	Riscos e pontos de atenção	

SOBRE ESTE DOCUMENTO

Descreve a arquitetura tal como implementada no código, não a intenção de projeto. Onde código e expectativa divergem, o texto registra a realidade do código e sinaliza o ponto.

01 Visão geral e stack tecnológico

O FlowDesk é uma plataforma de automação low-code para a IRKO. O usuário descreve um processo, conciliar duas planilhas ou extrair dados de um PDF, por exemplo, e um assistente de IA conduz uma entrevista, monta o fluxo e escreve o código Python. O resultado é uma automação que pode ser testada, versionada e publicada como uma aplicação interna com formulário próprio.

Frontend

React 18 + TypeScript + Vite
Tailwind CSS, React Router, Framer Motion
React Flow (canvas), Monaco (editor de código)
react-markdown (chat)

Backend

FastAPI (Python), Uvicorn
SQLAlchemy 2.x, Pydantic Settings
Streaming SSE no chat, WebSocket no monitor

Persistência

Postgres (Supabase) via psycopg v3
Fallback automático para SQLite local
Arquivos de runtime em disco, por projeto

Integrações

OpenAI (gpt-4o-mini) para o assistente
OCR local (pdfplumber, PyMuPDF, RapidOCR)
SMTP opcional para notificação de falhas

Os dez módulos de produto

Assistente de criação · Smart Chat · Editor de fluxo (canvas) · Código-fonte (Monaco) · Gerenciador de arquivos · Histórico de versões · Monitor de workflow · Controle de acesso · Configurações do projeto · App publicado.

02 Arquitetura de alto nível

A aplicação segue um modelo cliente-servidor clássico: uma SPA React conversa com uma API FastAPI por HTTP/JSON. A API concentra três responsabilidades, servir o console interno, executar automações em um runtime próprio e expor os apps publicados ao público autorizado.

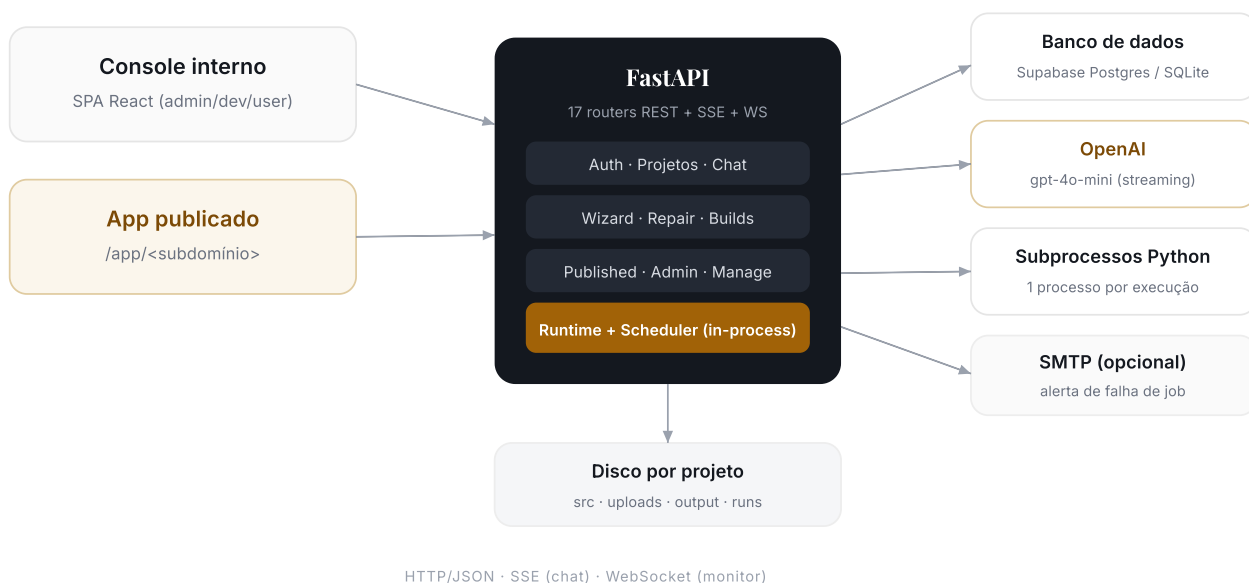


FIGURA 1. COMPONENTES PRINCIPAIS E SUAS DEPENDÊNCIAS

PONTO-CHAVE

O backend é um único processo FastAPI. O runtime de execução e o scheduler de jobs rodam dentro desse processo, como tarefas asyncio. Não há serviço de fila externo nem orquestrador de containers.

03 Frontend: os dois modos de uso

O console foi desenhado para duas audiências distintas, com gate por papel definido em `App.tsx`.

Modo assistente

TODOS

Caminho do usuário não técnico. Conversa em linguagem natural, responde à entrevista guiada e testa o resultado. Sem contato com código.

Páginas: Console, Chat, Wizard, App publicado.

Modo avançado

ADMIN / DEV

Editor de fluxo (React Flow), editor de código (Monaco), gerenciador de arquivos, versões, logs e monitor de workflow.

Protegido por `EditorGate`: usuário comum é redirecionado ao assistente.

Páginas (front/src/pages)

PÁGINA	FUNÇÃO	ACESSO
Login	Autenticação por e-mail e senha	PÚBLICO
Console	Lista de automações, pastas, galeria de modelos	AUTENTICADO
Chat · Wizard	Criação e refino guiados por IA	AUTENTICADO
Editor	Canvas de fluxo + Monaco	ADMIN/DEV
WorkflowMonitor	Execuções em tempo real (WebSocket)	AUTENTICADO
Builds · Logs · Files	Versões, logs e arquivos do projeto	AUTENTICADO
Manage · Admin	Saúde das automações, usuários, uso de IA	ADMIN/DEV
PublishedApp	App publicado com formulário próprio	WHITELIST

04 Backend: API e routers

A API expõe rotas sob o prefixo `/api`. Cada router cobre um domínio. O bootstrap em `main.py` registra os 17 routers, cria os índices, aplica micro-migrações e sobe runtime e scheduler.

ROUTER	RESPONSABILIDADE
<code>auth</code>	Login, emissão de token JWT, usuário atual
<code>projects</code>	CRUD de projetos, pastas, stages, edges, arquivos-fonte
<code>chat</code>	Smart Chat: streaming OpenAI e ações pendentes (aprovar/rejeitar)
<code>wizard</code>	Assistente de criação guiado por IA (análise da intenção)
<code>repair</code>	Auto-reparo de código com IA ciente do ambiente
<code>executions</code>	Disparo e consulta de execuções
<code>builds</code>	Versões imutáveis (snapshot) e restauração (rollback)
<code>published</code>	App público: info, login, formulário, submit, download
<code>realtime</code>	WebSocket do monitor de workflow
<code>dashboard</code> · <code>manage</code>	Métricas e gestão de saúde das automações
<code>admin</code>	Gestão de usuários e papéis, uso de IA
<code>filemanager</code> · <code>settings</code> · <code>hooks</code> · <code>templates</code>	Arquivos, configurações, webhooks, galeria de modelos

AUTENTICAÇÃO

O console usa token JWT (expiração de 7 dias). O papel efetivo combina a coluna `users.role` com listas de e-mail em variáveis de ambiente (`ADMIN_EMAILS` / `DEV_EMAILS`).

05 Persistência e modelo de dados

A seleção do banco é automática em `database.py` : se `SUPABASE_DB_URL` (ou `DATABASE_URL`) estiver definida, usa Postgres com o driver `psycopg` v3 e `pool_pre_ping` ; caso contrário, cai para SQLite local. O mesmo conjunto de modelos SQLAlchemy serve aos dois bancos.

SEPARAÇÃO IMPORTANTE

O código-fonte das automações vive em linhas `SourceFile` no banco, por isso persiste e versiona. Já os arquivos de execução, uploads, saídas e o input/output de cada run, vivem em disco sob `back/storage/<projeto>/` , materializados para a pasta `src` a cada execução, junto do `flowdesk_sdk.py` .

MULTI-TENANT POR ORG

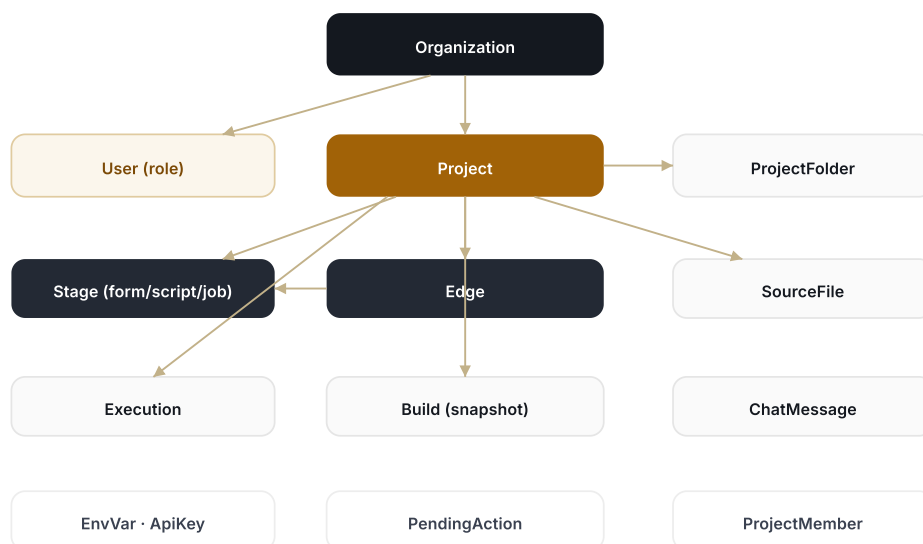


FIGURA 2. MODELO DE DADOS, PRINCIPAIS ENTIDADES E RELAÇÕES

Demais tabelas: Role, Connector, DataTable/DataRow (tabelas internas do projeto), todas vinculadas a um projeto.

06 Runtime de execução de automações

O coração da execução é o `RuntimeManager` (`runtime/runner.py`): uma fila FIFO asyncio com um worker. Cada stage do tipo script ou job vira uma execução enfileirada; o worker a executa em um **subprocesso Python isolado**, com as variáveis de ambiente do projeto injetadas e o código materializado em disco.

Ciclo de uma execução



FIGURA 3. CICLO DE VIDA DE UMA EXECUÇÃO

Características relevantes

- **Isolamento por subprocesso:** código de terceiros roda fora do processo da API; crash ou timeout não derruba o backend.
- **SDK injetado:** `flowdesk_sdk.py` oferece `get_input`, `set_output`, `get_file`, `output_path`, `log`, `extract_document` (OCR), `progress` (linha do tempo da execução), `get_table` (tabelas internas do projeto, como regras De/Para) e `read_table` (xlsx/xls/csv, tolerante a xls legado do Domínio via Excel da máquina).
- **Linha do tempo e tabelas:** a cada execução o runtime materializa as tabelas do projeto em `tables.json` e expõe os eventos de `progress()` (gravados em `progress.jsonl`) no polling da execução, alimentando a narração em tempo real no console e no app publicado.
- **Encadeamento:** ao concluir com sucesso, o runtime dispara os stages script/job seguintes ligados por `Edge`. Formulários são portões do usuário e não encadeiam sozinhos.
- **Resiliência de cold start:** falha intermitente de import de numpy/pandas no Windows dispara uma única nova tentativa.
- **Notificação:** job agendado que falha aciona e-mail SMTP, se configurado.

Scheduler de jobs

O `Scheduler` roda no mesmo processo, com tick a cada 30 segundos. Varre stages do tipo job habilitados, compara a última execução com o agendamento (intervalo ou diário) e enfileira quando vencido. Usar a última execução como marca permite sobreviver a reinícios.

07 Camada de inteligência artificial

A IA usa a OpenAI (modelo `gpt-4o-mini`). Sem chave configurada, a plataforma opera em modo simulado, com respostas determinísticas de demonstração. A IA aparece em três pontos.

Smart Chat

Streaming token a token. O modelo emite texto e, ao final, um bloco cercado `flowdesk-actions` com `questions` (perguntas) e `actions` (ações propostas).

Assistente de criação

Conduz uma entrevista de descoberta, uma pergunta por vez, antes de construir. Só pergunta o que muda a solução.

Auto-reparo

Diante de erro de execução, propõe correção de código ciente do ambiente real, bibliotecas instaladas e contrato do SDK.

Aprovação humana

Toda ação proposta vira uma `PendingAction` que o usuário aprova ou rejeita antes de aplicar. A IA não altera o projeto sozinha.

Classificação contábil

Em automações como extrato bancário para o Domínio, a IA sugere a conta contábil de cada grupo de histórico. Payload mínimo: só o texto do histórico e as contas candidatas saem da máquina, nunca valores, saldos ou CNPJ.

HÍBRIDO COM APRENDIZADO

A classificação contábil combina regras De/Para do projeto (que entram pré-aprovadas), sugestão de IA (que exige confirmação) e escolha manual, numa tela de revisão com semáforo. Cada correção vira regra para a próxima execução, então o esforço de revisão cai mês a mês. As regras vivem em

`DataTable` por projeto.

SEGURANÇA DA IA

O protocolo de ações estruturadas, bloco `flowdesk-actions` com aprovação humana, é o mecanismo central de segurança: o modelo sugere, a pessoa decide, o sistema aplica.

OCR roda localmente (pdfplumber para PDF com texto, PyMuPDF para rasterizar escaneados, RapidOCR para reconhecimento), com validação em três etapas, confiança, automação e revisão humana. A revisão humana editada realimenta o reprocessamento via campo `_texto_corrigido`.

08 Apps publicados e controle de acesso

Uma automação publicada vira um app independente em `/app/<subdomínio>`, com formulário próprio gerado a partir dos stages do tipo form. O usuário final preenche, envia, acompanha o processamento e baixa o resultado, sem ver o console.

Dois níveis de acesso

CAMADA	MECANISMO
Console interno	JWT, papel admin/dev/user, gate de modo avançado
App publicado	Por projeto: whitelist de e-mails ou domínio permitido; token próprio do app

FLAG TEMPORÁRIA

`PUBLIC_APPS_OPEN` habilita apps publicados sem login (qualquer pessoa com o link executa). É um ajuste de desenvolvimento. Desligar para restaurar o controle por whitelist ou domínio.

Versões e rollback

Cada publicação gera um `Build` imutável com snapshot do código (hash curto). É possível restaurar o código de uma versão anterior, o que faz o upsert dos arquivos do snapshot e remove os que não existiam nele.

09 Fluxos principais (ponta a ponta)

A · Criação pelo assistente

1. Usuário descreve o processo no Chat e anexa um arquivo de exemplo.
2. Cria-se um projeto; a IA conduz a entrevista.
3. A IA propõe ações como pendências.
4. Usuário aprova; o sistema cria stages, edges e arquivos-fonte.
5. A IA dá nome amigável ao projeto a partir do pedido real.

B · Execução de uma automação

1. Gatilho cria uma Execution (teste, app, job ou webhook).
2. O runtime materializa o código e roda o subprocesso isolado.
3. Saída e status são persistidos e enviados ao monitor por WebSocket.
4. Em sucesso, os stages seguintes ligados por Edge são encadeados.

C · Uso de um app publicado

1. Usuário autorizado abre o app e preenche o formulário.
2. O submit cria uma execução; a tela faz polling até concluir.
3. Resultado é exibido; revisão de OCR bloqueia o download até confirmação.

D · Job agendado

1. O scheduler detecta um job vencido e enfileira a execução.
2. Em falha, dispara e-mail de alerta, se SMTP configurado.

10 Decisões de arquitetura e trade-offs

DECISÃO	MOTIVAÇÃO	TRADE-OFF
Subprocesso por execução não VM, não container	Isolar código de terceiros; crash não derruba a API; simplicidade operacional	Cold start de bibliotecas a cada run; custo de inicialização por execução
Runtime e scheduler in-process	Zero infraestrutura extra; fácil de operar em uma máquina	Worker único (concorrência 1): execuções enfileiram sob carga; não escala horizontalmente
Código-fonte no banco SourceFile	Persistência, versionamento e snapshot de builds	Materialização em disco a cada execução
Postgres com fallback SQLite	Produção no Supabase, desenvolvimento sem dependência	Micro-migrações precisam cobrir os dois dialetos
IA com aprovação humana	Segurança: o modelo nunca aplica mudança sozinho	Passo extra de aprovação no fluxo

ESCLARECIMENTO

Sobre a percepção de que a aplicação cria várias máquinas virtuais: não cria. Cada automação roda como um subprocesso efêmero do mesmo interpretador Python. O custo real percebido como peso é o cold start das bibliotecas por execução somado à serialização do worker único, não virtualização.

11 Riscos e pontos de atenção

PONTO	SEVERIDADE	ENCAMINHAMENTO SUGERIDO
Worker único limita throughput de execuções	MÉDIO	Pool de workers persistentes (processos reutilizados que já carregaram as libs), mantendo o isolamento
Cold start de pandas/numpy por execução	MÉDIO	Medir startup x trabalho útil antes de reengenharia; o pool de workers resolve junto
<code>PUBLIC_APPS_OPEN</code> deixa apps sem login	ALTO EM PRODUÇÃO	Desligar e restaurar whitelist/domínio antes de exposição externa
Visibilidade por organização, sem dono por projeto	EVOLUÇÃO	Introduzir propriedade e compartilhamento por projeto
Segredos de banco e chave de IA	OPERACIONAL	Manter em <code>.env</code> fora do versionamento; rotacionar credenciais expostas

Resumo executivo

A arquitetura é coerente e enxuta para a escala atual: um backend único, execução isolada por subprocesso, IA com aprovação humana e apps publicados com controle de acesso. Os limites conhecidos, concorrência de execução e cold start, são de desempenho, não de correção, e têm caminho de evolução claro sem reescrever o núcleo. As prioridades antes de exposição externa são desligar a abertura pública dos apps e introduzir propriedade por projeto.